8.DMA

1. DMA 的硬件原理

1.1 DMA 是什么

DMA（Direct Memory Access，直接存储器访问）可以在**不让 CPU 逐个搬运数据**的情况下，在**外设**和**内存**之间传输数据。

例如，用串口接收 100 个字节：

- 不用 DMA：UART 每收到一个字节，CPU 都要读取一次 UART 数据寄存器，再写入数组。
- 使用 DMA：CPU 只需提前告诉 DMA“从 UART 数据寄存器搬 100 个字节到这个数组”，之后 DMA 自动完成搬运。

可以把**CPU 理解为管理员**，**DMA 理解为搬运工**：CPU 负责填写搬运单，DMA 负责实际搬运，完成后再通知 CPU。

1.2 DMA 的基本数据通路

代码块

```
外设数据寄存器  <---- DMA ---->  SRAM 中的数组
   例如 UART_DR                   例如 rx_buf[]
```

DMA 一次传输需要知道以下信息：

1. **源地址和目标地址**：**从**哪里读取数据和**往**哪里写数据。

2. **传输数量**：一次搬运**多少个**数据单元。

3. **数据宽度**：每个数据单元是 8、16 还是 32 位。

- **注意**：需要同时设置**外设存储宽度**和**存储器数据宽度**

4. **地址是否递增**：每搬一次后，**源地址**或**目标地址**是否移动。

- 规则：若传输的是**“连续数据块”（如数组）**，对应地址需设为「增量模式（Enable）」；
- 若传输的是**“单个固定地址”（如外设寄存器）**，对应地址设为「非增量（Disable）」。

例：ADC→数组：外设地址（ADC_DR 寄存器）固定（Disable），内存地址（数组）增量（Enable）；
数组→UART：内存地址（数组）增量（Enable），外设地址（USART_DR 寄存器）固定（Disable）。

1.3 常见传输方向

方向	典型用途	源地址	目标地址
外设到内存	ADC 采样、UART/SPI 接收	固定的外设数据寄存器	通常递增的数组地址
内存到外设	UART/SPI 发送、DAC 输出	通常递增的数组地址	固定的外设数据寄存器
内存到内存	数组复制、数据整理	通常递增	通常递增

外设寄存器通常只有一个**固定地址**，因此外设地址一般**不递增**。**数组中的元素**位于**连续地址**，因此内存地址通常**递增**。

1.4 一次 DMA 传输如何发生

以“UART 接收数据到数组”为例：

1. CPU 配置 UART 和 DMA。
2. CPU 告诉 DMA：外设地址是 UART 接收数据寄存器，内存地址是 `rx_buf`，数量是 100。
3. CPU 使能 UART 的 DMA 接收请求，并启动 DMA。
4. UART 每收到一个字节，就向 DMA 发出一次请求。
5. DMA 读取 UART 数据寄存器，并写入 `rx_buf` 当前位置。
6. DMA 自动增加内存地址，同时把剩余数量减 1。
7. 数量减到 0 时，DMA 置位“传输完成”标志，并可触发中断。

注意：DMA 并不会让 UART、ADC 或 SPI 自己开始工作。**它只负责搬数据**。外设仍要被正确配置和启动。

2. DMA 的核心概念

2.1 通道

我们现在常用的单片机有两个DMA，分别是DMA1（基本型）和DMA2（增强型），而每个DMA又分别有8个**数据流，通道数量与单片机型号有关**，每个数据流都可以构建起一条 `源` 和 `目标` 间的**单向传输通路**

- 不同通道之间可以**定义优先级**
- 每个通道可以**自定义传输数量**的多少，最大不超过65535

注意：每个DMA通道对应一个或多个**外设请求**

核心规则是：“**一个外设的某个功能，只能使用固定的DMA通道；一个DMA通道同一时间只能服务一个外设请求**”。（前者在使用cubemx时不必在意，cubemx已经为我们锁定了每个外设所对应的DMA数据流）

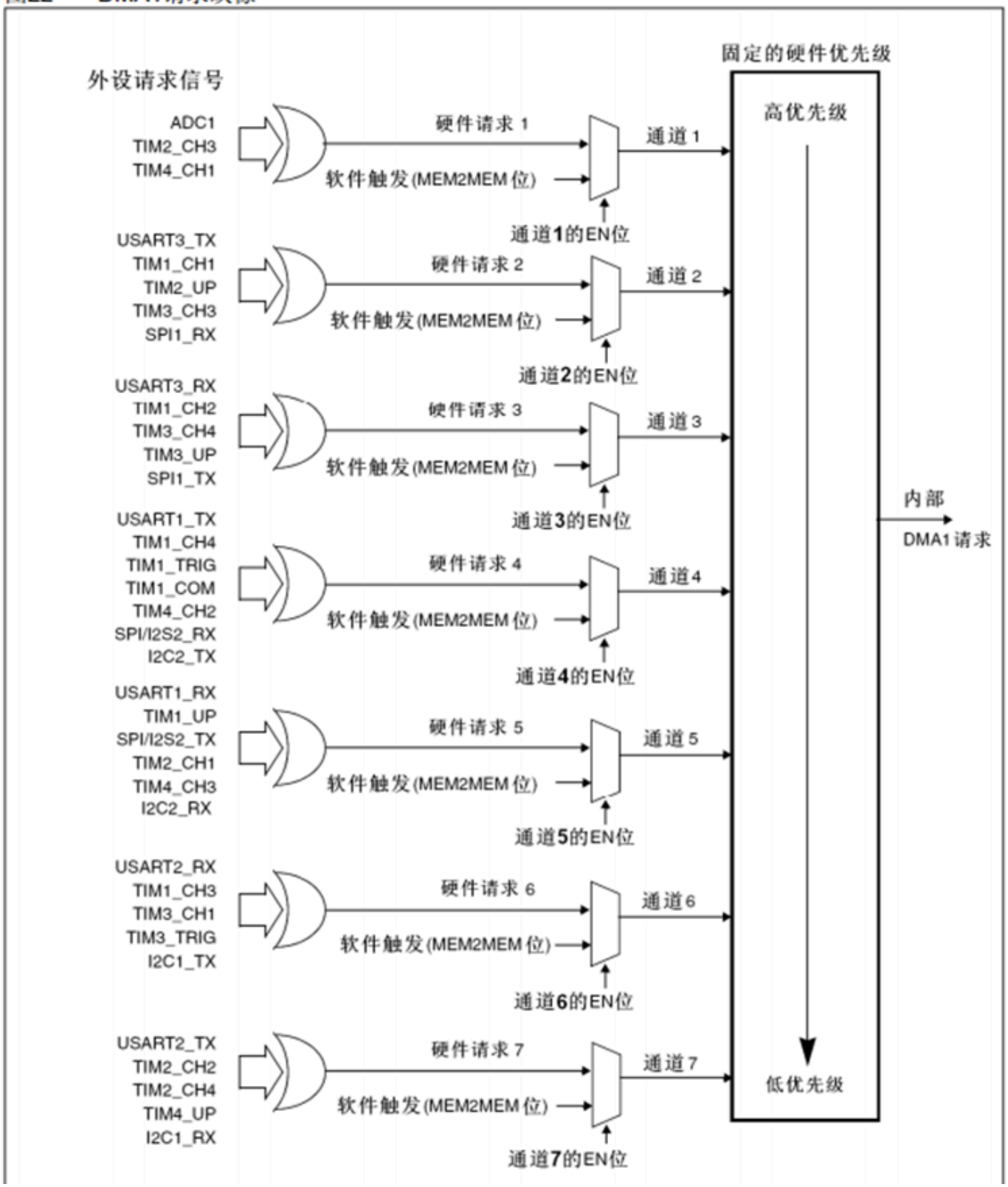
2.2 优先级

常见优先级为 **Low、Medium、High、Very High**。

- 注意：优先级只影响 DMA 请求的**仲裁顺序**，通常不会中断已经开始的一个总线传输。

它也**不等价于** NVIC 中断优先级：前者决定谁先使用 DMA，后者决定 CPU 先响应哪个中断。

图22　DMA1请求映像



2.3 模式

2.3.1 Normal（普通模式）

传输数量减到 0 后自动停止，产生一次传输完成事件。

适合固定长度的一帧数据、一次性发送或一次性 ADC 缓冲区采集。

下一帧必须**重新调用 HAL 函数启动**。

2.3.2 Circular（循环模式）

传输完成后**自动把地址和计数恢复到循环起点**，继续响应请求。

适合连续 ADC、音频、传感器采样和串口环形接收。

CPU 必须明确“DMA 正在写哪一段”，不能在 DMA 仍写入时处理同一段数据。

循环模式通常可以利用半传输（HT）和传输完成（TC）事件，把一个缓冲区分成前后两半：

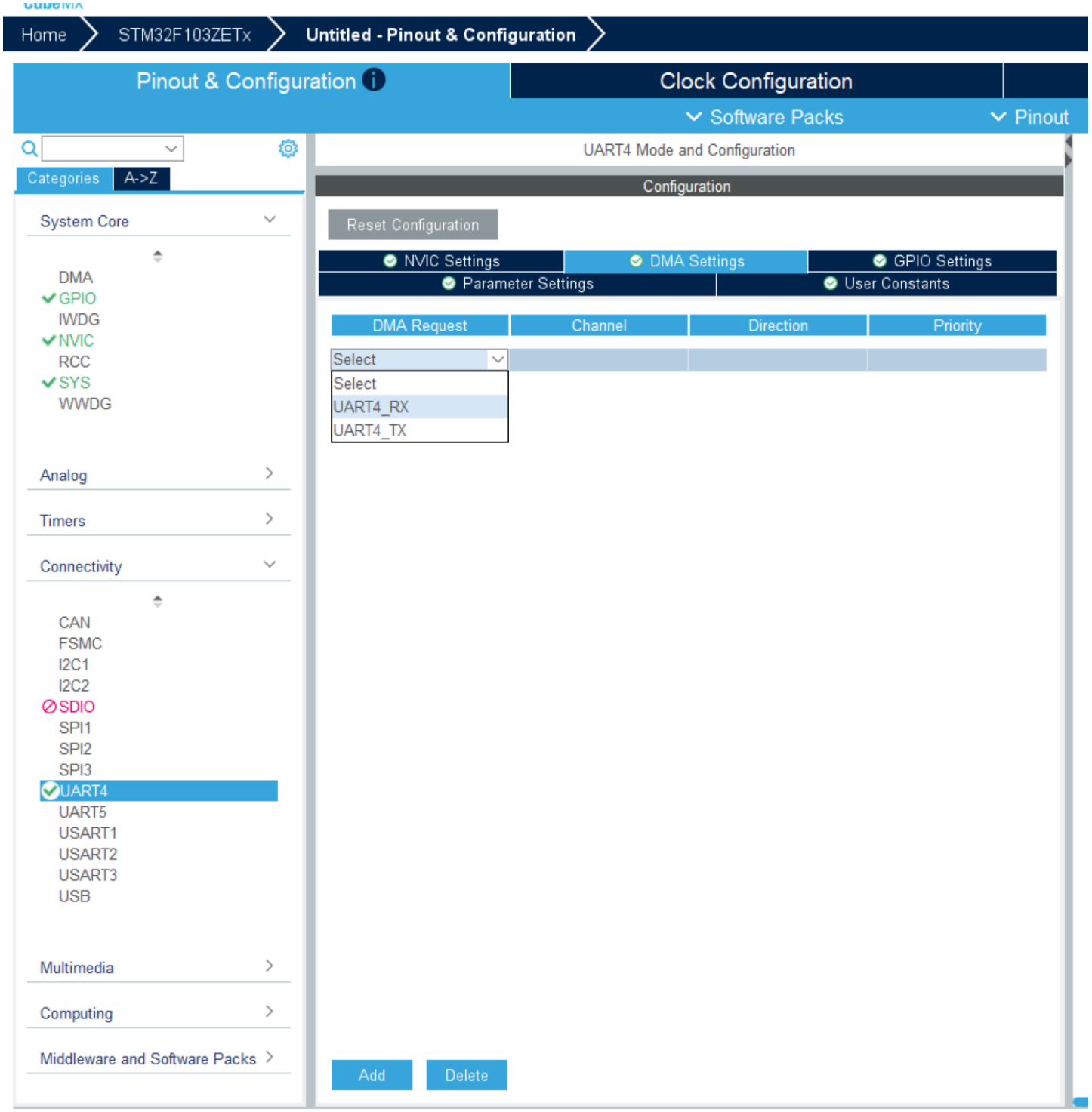
```
|---- 前半区 ----|---- 后半区 ----|
        HT                TC
```

3. CubeMx配置

(以STM32F103ZETx的串口4的Rx为例,前置调试频率等配置不再赘述)

3.1添加DMA

选择DMA Settings，点击下方add按钮添加dma，并选择Rx or Tx



3.2 优先级，模式，数据宽度配置

点击 `Priority` 栏可以选择优先级，这就是上文提到的仲裁器判断优先级的依据

具体到寄存器层面，这个选项将改变DMA_CCRx寄存器中的**PL[1;0]位，**

- 00：低
- 01：中
- 10：高
- 11：最高

`Mode` 栏中可以选择DMA模式，有正常模式和循环传输模式

由DMA_CCRx寄存器中的**CIRC位控制**

- 0：不执行循环操作
- 1：执行循环操作

`Data Width` **则表示上文所讲的*数据宽度***

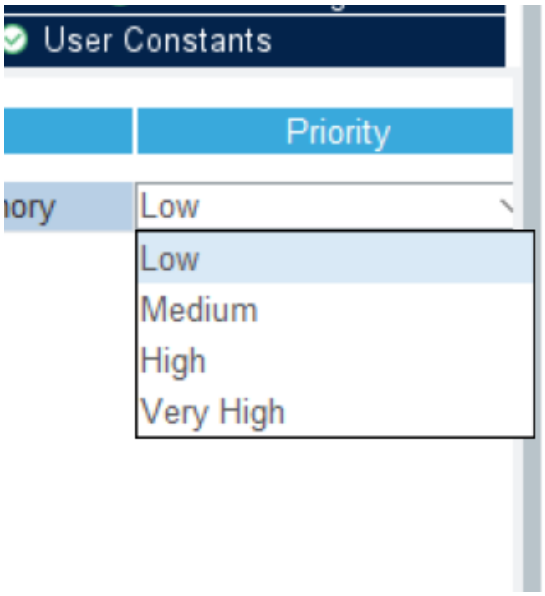
由DMA_CCRx寄存器中的**MSIZE[1:0]位和PSIZE[1:0]位控制**

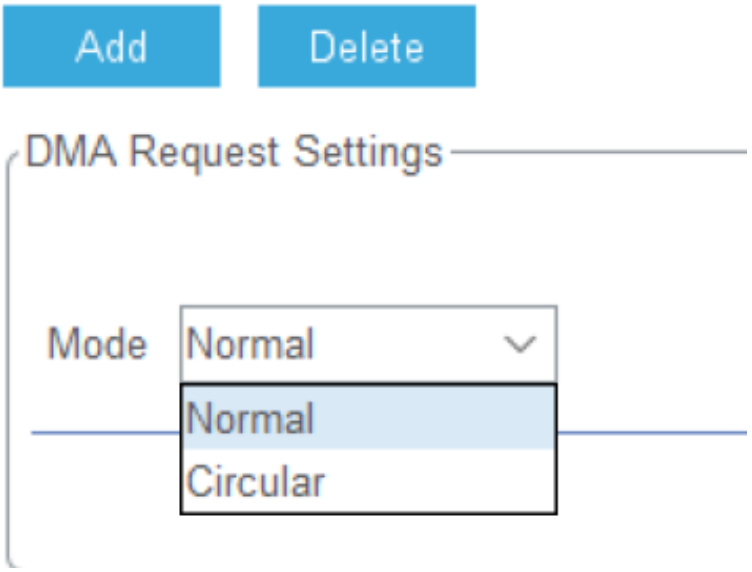
MSIZE[1:0]：存储器数据宽度 (Memory size)

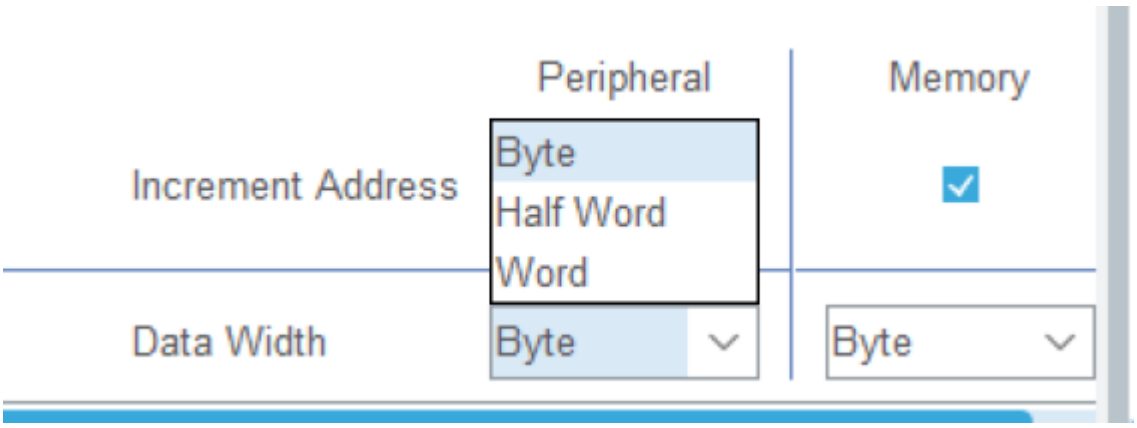
- 00：8位
- 01：16位
- 10：32位
- 11：保留

PSIZE[1:0]：外设数据宽度 (Peripheral size)

- 00：8位
- 01：16位
- 10：32位
- 11：保留







3.3NVIC配置

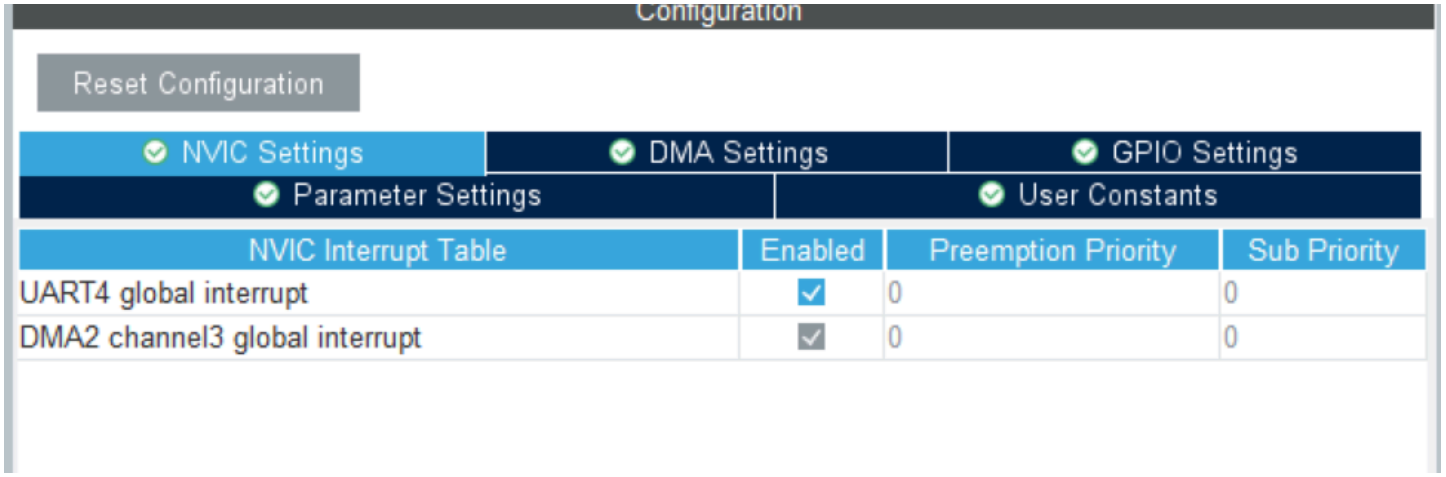
在`NVIC Settings`栏中，开启**串口全局中断**

这个中断包含了**半传输中断、传输完成中断**和**传输错误中断**

当中断发生时，程序会运行cubemx自动生成的**中断处理函数**，但这个函数中不执行任何操作

此函数为一个**弱函数**，可在它处进行重写，进而完成数据处理或其他操作

具体见HAL库配置中的**中断回调函数**



由DMA_CCRx寄存器中的**TEIE位和HTIE位以及TCIE位控制**

TEIE：允许传输错误中断 (Transfer error interrupt enable)

- 0：禁止TE中断
- 1：允许TE中断

HTIE：允许半传输中断 (Half transfer interrupt enable)

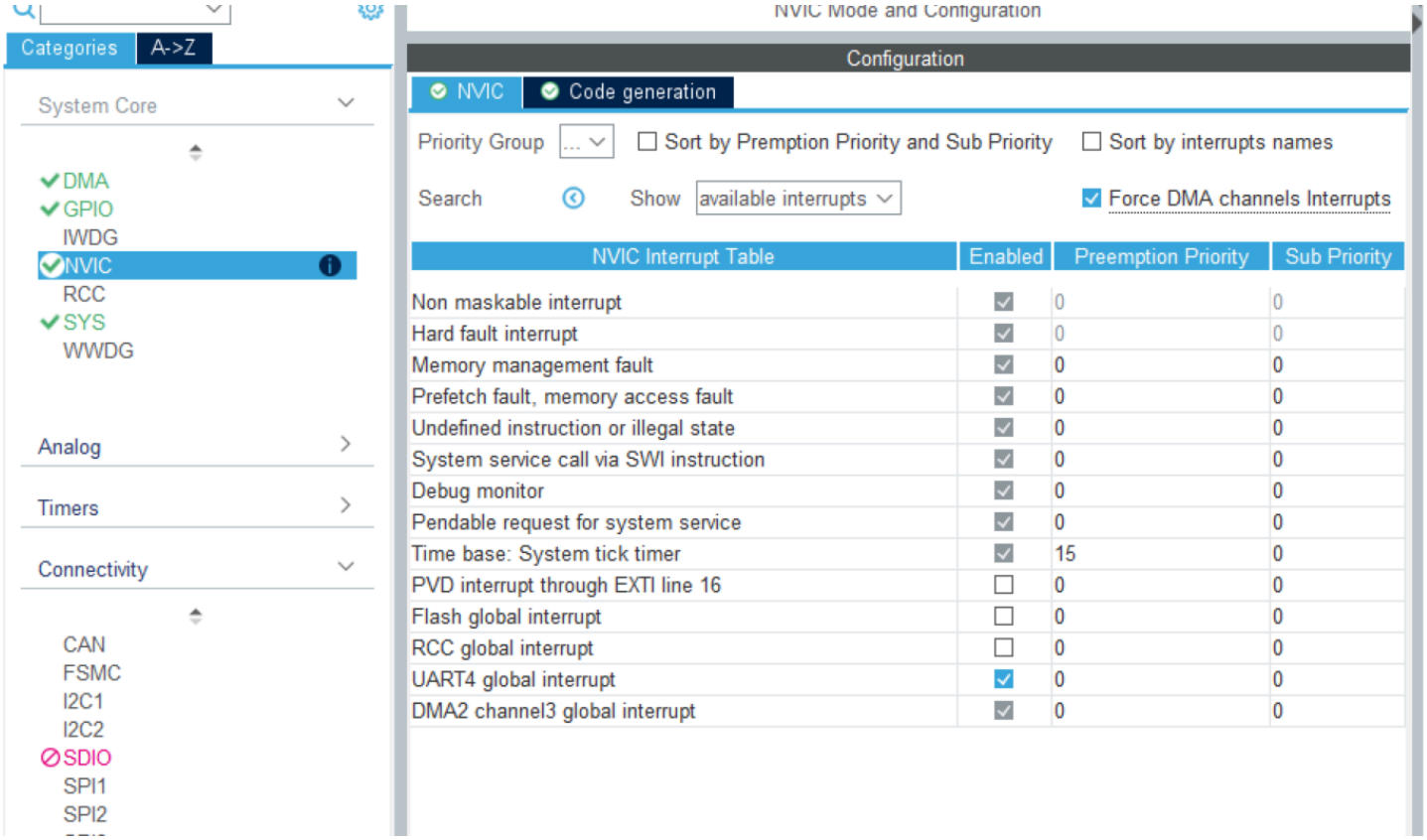
- 0：禁止HT中断
- 1：允许HT中断

TCIE：允许传输完成中断 (Transfer complete interrupt enable)

- 0：禁止TC中断
- 1：允许TC中断

如果想**关掉DMA的中断**，则可以在`NVIC`的右上角取消勾选`Force DMA channels interrupts`，然后DMA中断也就可以取消勾选了。

在这里关闭中断时，**TEIE位和HTIE位以及TCIE位将被同时置0，即全部失能**



4. HAL库函数

1.__HAL_DMA_GET_FLAG(__HANDLE__, __FLAG__)

- __HANDLE__ 指**DMA句柄**

例如

例如 CubeMX 生成的 **DMA 句柄**：

```
代码块
1   DMA_HandleTypeDef hdma_usart1_rx;
```

- __FLAG__ 指具体需要获取的**标志位**，

DMA 状态标志位，用于判断某个 DMA Stream 是否发生了某种事件。

事件	标志宏	含义
传输完成	DMA_FLAG_TCIFx_x	本次传输全部完成
半传输完成	DMA_FLAG_HTIFx_x	传输完成一半
传输错误	DMA_FLAG_TEIFx_x	DMA 访问发生错误

```
代码块
1   #define DMA_FLAG_GL1                        0x00000001U
```

```
#define DMA_FLAG_TC1                    0x00000002U
#define DMA_FLAG_HT1                    0x00000004U
#define DMA_FLAG_TE1                    0x00000008U
#define DMA_FLAG_GL2                    0x00000010U
#define DMA_FLAG_TC2                    0x00000020U
#define DMA_FLAG_HT2                    0x00000040U
#define DMA_FLAG_TE2                    0x00000080U
#define DMA_FLAG_GL3                    0x00000100U
#define DMA_FLAG_TC3                    0x00000200U
#define DMA_FLAG_HT3                    0x00000400U
#define DMA_FLAG_TE3                    0x00000800U
#define DMA_FLAG_GL4                    0x00001000U
#define DMA_FLAG_TC4                    0x00002000U
#define DMA_FLAG_HT4                    0x00004000U
#define DMA_FLAG_TE4                    0x00008000U
#define DMA_FLAG_GL5                    0x00010000U
#define DMA_FLAG_TC5                    0x00020000U
#define DMA_FLAG_HT5                    0x00040000U
#define DMA_FLAG_TE5                    0x00080000U
#define DMA_FLAG_GL6                    0x00100000U
#define DMA_FLAG_TC6                    0x00200000U
#define DMA_FLAG_HT6                    0x00400000U
#define DMA_FLAG_TE6                    0x00800000U
#define DMA_FLAG_GL7                    0x01000000U
#define DMA_FLAG_TC7                    0x02000000U
#define DMA_FLAG_HT7                    0x04000000U
#define DMA_FLAG_TE7                    0x08000000U
```

2.__HAL_DMA_CLEAR_FLAG(__HANDLE__, __FLAG__)

参数与__HAL_DMA_GET_FLAG(__HANDLE__, __FLAG__)相同

作用为**清除某个标志位**

3.__HAL_DMA_ENABLE_IT(__HANDLE__, __INTERRUPT__)

- __HANDLE__　　指**DMA句柄**
- __INTERRUPT__　表示要打开的 **DMA 中断类型**，是一个**中断掩码**，不是 IRQ 编号。

常见取值：

参数	含义
DMA_IT_TC	传输完成中断
DMA_IT_HT	半传输完成中断
DMA_IT_TE	传输错误中断

4.__HAL_DMA_DISABLE_IT(__HANDLE__, __INTERRUPT__)

参数与__HAL_DMA_ENABLE_IT(__HANDLE__, __INTERRUPT__)相同

表示**失能某个中断**

5.HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)

用于让**指定 UART 通过 DMA 异步发送一段数据**

- UART_HandleTypeDef *huart 表示**串口句柄**

例如 CubeMX 生成了：

```
代码块
1   UART_HandleTypeDef huart1;
```

调用时传入它的地址：

```
代码块
1   &huart1
```

- uint8_t *pData 待发送数据的**首地址**，也就是 DMA 的**内存源地址**
- uint16_t Size 要发送的**数据长度**，单位是**字节数**。

完整示例：

```
代码块
1   HAL_UART_Transmit_DMA(&huart1, tx_data, 5);
```

6.HAL_UART_Receive_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)

用于**让指定 UART 通过 DMA 异步接收一段数据**

参数与HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)相同

7.HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)

UART DMA 接收完成回调函数。

它的作用不是启动接收，而是通知用户：

> UART DMA 已经按照指定长度接收完一批数据，可以处理接收缓冲区了。

- 典型调用流程

代码块

```
HAL_UART_Receive_DMA()
         |
         v
UART 接收数据
         |
         v
DMA 传输完成
         |
         v
DMA IRQ Handler
         |
         v
HAL_DMA_IRQHandler()
         |
         v
HAL_UART_RxCpltCallback()
```

前面提到HAL_UART_RxCpltCallback()是一个**虚函数**，可以在别处**重写**来拿实现相应功能

代码块

```
__weak void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  /* Prevent unused argument(s) compilation warning */
  UNUSED(huart);
  /* NOTE: This function should not be modified, when the callback is needed,
           the HAL_UART_RxCpltCallback could be implemented in the user file
```

```
7       */
8   }
```

具体位置在**Drivers\STM32F1xx_HAL_Driver\Src\stm32f1xx_hal_uart.c的第2622行**

8.HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)

与HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)类似

9.HAL_UART_DMAPause(UART_HandleTypeDef *huart)

暂停串口DMA

10.HAL_UART_DMAResume(UART_HandleTypeDef *huart)

恢复串口DMA

11.HAL_UART_DMAStop(UART_HandleTypeDef *huart)

结束串口DMA

注：关于Pause和Stop的区别

函数	作用	DMA 当前进度	后续操
HAL_UART_DMAPause()	暂时暂停 DMA 请求	保留当前地址和剩余数量	用 HAL_UART_DMA
HAL_UART_DMAStop()	终止当前 DMA 传输	通常中止并清理当前传输	需要重新调用发

示例：如果我想在收到一条消息后向上位机发送消息

1.cubemx配置

这里我使用的是串口4的dma

配置与第三节配置类似

2.代码

现**定义接收与发送缓冲区**，也就是一个数组，类型是**uint8_t**

然后主函数中**开启串口的dma接收**

```
/* USER CODE BEGIN 2 */
HAL_UART_Receive_DMA(&huart4,rxbuff, Size: 10);
/* USER CODE END 2 */
```

这样我们就成功开启了数据的**dma接收**

那么我们如何使用回调函数呢?

这里以**完成回调**为例

在**主函数外**重写HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)函数

```
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
  if (huart->Instance == UART4)
  {
    HAL_UART_Transmit_DMA(&huart4,txbuff, Size: sizeof(txbuff));
  }

}

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
```

在回调函数中先判断是否是你使用的串口（**uart4**）产生的中断，然后在代码块中完成**dma发送**

这样我们就实现了**在接收完消息后，向上位机发送消息**

5. 练手任务

1. **通过串口调试助手，使用串口DMA接收上位机发送的消息并且再回传给上位机，每次回传完成让小灯改变亮灭状态**

大致思路：

1. 正确配置cubemx中的相关设置（时钟树，GPIO，串口，DMA），尤其要注意SYS中的debug选项不要忘了改
2. 要判断是否发送和接收完成，肯定需要中断，所以打开DMA中断
3. 重写DMA接收完成回调函数和DMA发送完成回调函数，达到目标

2. **写一个DMA双缓冲区接收，通过串口调试助手从上位机给单片机发送消息，在切换缓冲区向上位机发送当前使用的缓冲区编号（最好能通过debug查看当前被使用的缓冲区）**

原本的DMA接收只有一个缓冲区，可能存在处理数据不及时或处理数据较慢而导致后续数据无法运输而丢失的情况，此时双缓冲区就显得很有必要

大致思路一：

1. 正确配置cubemx
2. 创建两个缓冲区（两个数组之间切换/或者一个大数组前后切换）
3. 重写接收回调函数，在接收完成后切换缓冲区，同时处理当前缓冲区中的内容（改变小灯亮灭并清空当前缓冲区），如此往复，可以提高传输速度

提示：**普通模式**和**循环模式**所需代码细节不同

需要**注意**的是，为了防止DMA在我们处理接收内容时还在传输，覆盖掉部分数据，让数据出错（或者别的什么蛊），一定要在处理数据前先停止DMA传输，**等数据处理完成后**再重新启动DMA的传输（要是用普通模式就没有这个问题，结合上面所说的普通模式和循环模式的区别想一想为什么吧）

6.思考题

1. **说明启用串口DMA时cubemx的相关配置过程**
2. **说明DMA中断发生的整个流程**
3. **DMA 的 "地址增量" 配置有什么意义？外设地址和内存地址是否都需要开启增量？为什么？**

4. 在 DMA 循环模式下，若缓冲区大小为 128 字节，当第 130 个字节到来时，缓冲区中的数据会如何变化？此时 CPU 读取数据需要注意什么？

By ssy

QQ: 3631754124